

WEEK 06

资产化数据工厂

DATA ASSET FACTORY — Orchestration · Backfill · Lineage · Runbook

01 Asset

范式转换

从任务流
到资产流

02 Partition

原子单位

时间 / 维度
复合分区

03 Backfill

工程化补数

4 阶段
3 层幂等

04 Lineage

运行时血缘

OpenLineage
自动血缘

05 Runbook

应急 + 协作

5 要素 →
Skill Pack

把数据从“脚本输出”升级为“工程化资产”——可寻址、可版本、可责任、可观测、可重建

HOW WE WILL LEARN

本周怎么学:5 节课 · 一个真项目 · 一份能上岗的能力

原理

建立判断

- 从「任务流」走到「资产流」的工程动因
- Iceberg V3 / DuckLake / OpenLineage 2026

现状

- 资产 / 分区 / 幂等 / 血缘的工程对象拆解
- 不堆 PPT,只立判断——5 节课每节 5 个判断点

实战

可上仓库

- 全周贯穿 OmniSupport Copilot 项目
- 本周产出:assets / partitions / backfill_jobs
/lineage / runbook / postmortem
- 在 GitHub 仓库里直接 walk-through
- Dagster Cloud + DuckDB 一键起跑

协作 + 演练

团队级

- 资产命名 / Owner / SLO 团队约定
- Branch Deployments + slim CI 变更隔离
- Game Day 演练把 Runbook 跑成肌肉记忆
- Postmortem 模板 → 沉到知识库
- 为 Week 7 非结构化数据交付稳定上游

L01

LESSON 01

FROM TASK FLOW TO ASSET FLOW

从任务流到资产流:数据工厂的范式转换

做过 Airflow / DolphinScheduler / Argo 的同学,半夜被 DAG 报警叫起来过的多半有同样的体会——

你改了一行代码,下游 50 张表里到底哪几张受影响,你说不清楚;
上游补数补了一周,下游数据还在用旧版本。

这一讲不讲调度引擎对比,讲清一件事:
为什么「任务流」这套思维在 AI 数据时代必须升级成「资产流」。

KEY THESIS

资产化不是工具升级,是数据团队的范式换挡—— 从「我跑了什么」转向「我交付了什么」

Airflow 这套东西最开始是任务调度器,它只关心一件事:这个 DAG 跑没跑。
但 AI 时代下游 RAG / Agent 真正在问的问题完全是另一组:

- 我现在拿到的 customer_dim,是哪一个版本?
- 最近一次更新是什么时候?
- 这张表谁负责?出问题该找谁?
- 要回退到上一个版本,我得花多久?

前面是脚本视角,后面是产品视角。

资产化就一句话——把数据当产品管理:可寻址、可版本、可责任、可观测。

老司机说 → Airflow 没错。错的是用 Task 这个抽象去服务一个需要回答「这是哪一版」的下游。

THREE PARADIGMS

三种数据编排范式——你团队在哪一档？

维度	脚本式 ETL	任务流(Airflow)	资产流(Dagster/dbt)
核心单位	Python 脚本	Task / Operator	Software-Defined Asset
首要关心	脚本跑没跑	DAG 跑通没跑通	资产现在是什么状态
故障定位	看日志,人肉排查	看 DAG 重试记录	看物化历史 + 血缘
影响分析	人肉评估,容易漏	看上下游 Task,粗	Asset Graph 一键渲染
补数	写专用脚本	配 Backfill 但要人肉判断	声明式 Partition 自动补
给 AI 的承诺	没有	弱(只到 Task 层)	强(每个分区有 snapshot)

不是 Airflow 不好,是它的核心抽象错位了——「Task」服务不了「我要拿到第 42 版的 silver」这种诉求。

WHY TASK FLOW BREAKS

任务流的 3 个隐藏代价——上线半年才会爆

COST · 补数

一次补数 ≈ 半周协调

- 改一个上游字段
- 人肉评估影响范围
- 写专用 backfill 脚本
- 通知 N 个下游团队各自补
- 补完了还要彼此校对

→ 真实数字:某厂 2025 年
一个季度做了 11 次补数,
平均每次占 1 个工程师 3 天

COLLAB · 协作

一次冲突 ≈ 一天扯皮

- 两个团队改同一张表
- 没有变更隔离机制
- PR 合到主分支才发现冲突
- 冲突修复 = 互相甩锅

→ 根因:
仓库里 dev / prod 共用
一套连接,改一处全炸

→ 解药(L5 会讲):
Branch Deployments + slim CI

TRUST · 信任

一次问询 ≈ 半月信任流失

- AI 团队:这表是新的吗?
- 业务团队:应该是吧?
- 数据团队:要看哪批数据。
- 没人能给确定答案

→ 后果:
AI 团队开始自己抓数据,
数据团队被绕过,
治理彻底崩塌

→ 这件事必须修——根本上修

这 3 个代价在脚本式 / 任务流时代是「可以忍」,在 AI 数据时代是「忍不住」——AI 系统对「数据这一刻是什么样」的精度要求,和 BI 不在一个量级。

FOUR ENGINEERING COMMITMENTS

资产化的 4 条军规——少一条都不算「真资产化」

01 Addressability · 可寻址

每份资产都有稳定 ID(namespace + key + version),可以被像 URL 一样引用。
不是「那个表」,是 support_ops.silver.ticket_fact@snap_2026_05_18_042。

02 Versioning · 可版本

每次物化产生一个不可变 snapshot。下游消费方可以锁到某个 snapshot_id。
回退就是切个指针,秒级完成——Iceberg 的 rollback_to_snapshot 是天然底座。

03 Ownership · 可责任

每份资产挂明确的 owner / SLO / criticality。
告警不是「群里 @everyone」,是按 routing 自动找到主负责人。MTTR 从小时压到分钟。

04 Observability · 可观测

物化历史 / 血缘 / 质量指标 / 变更日志全部可查。
资产页面像一个产品仪表盘,而不是黑盒 ETL——所谓「数据产品」的产品两个字,就指这个。

REAL ARTIFACT

OmniSupport 项目里的资产定义 —— 别只看好看,看每一行都有的工程含义

python · pipelines/assets/ticket_silver.py

```
from dagster import (
    asset, AssetIn, FreshnessPolicy,
    DailyPartitionsDefinition,
)

@asset(
    key_prefix=["support_ops", "silver"],
    name="ticket_fact",
    group_name="silver_support",
    partitions_def=DailyPartitionsDefinition(
        start_date="2026-01-01"),
    ins={"raw_ticket": AssetIn(
        key_prefix=["support_ops", "bronze"])},
    owners=["team:support-platform"],
    freshness_policy=FreshnessPolicy(
        maximum_lag_minutes=30),
    tags={
        "criticality": "P1",
        "contract": "tic.northstar.support_ticket.v3",
        "pii": "true",
    },
    metadata={
        "description": "工单事实表",
        "downstream_consumers": ["rag-faq", "support-kpi"],
    },
)

def ticket_fact(context, raw_ticket):
    df = clean_and_normalize(
        raw_ticket,
        partition_key=context.partition_key,
    )
    context.add_output_metadata({
        "row_count": len(df),
        "freshness_minutes": measure_lag(df),
        "contract_version": "v3",
        "snapshot_id": current_snapshot_id(),
    })
    return df
```

INSTRUCTOR NOTES · 老司机说

这段代码不长,但每一行都对得上前面的 4 条军规。

- key_prefix + name → 稳定 ID,这就是 Addressability。
- partitions_def → 每天独立物化,版本天然按分区切,这就是 Versioning 的根。
- owners + freshness_policy + tags → Ownership 的元数据,告警 routing 直接靠这个。
- add_output_metadata → 每次跑完留下「这一格用了什么、产出了什么」,Observability 的入口。

一个原则:asset 函数内部别再藏「我跑哪个分区」「我用哪个版本」这种动态判断——通通让框架来推。

SELF-CHECK

团队自检 5 题——超过 3 个答「不」,你团队还在脚本时代

Q1 能不能 30 秒内回答「customer_dim 当前是哪一版?上一次更新几点?」

答「需要登服务器看日志」= 没有 Observability。

Q2 改一个核心字段前,你能不能先看到「下游哪 23 张表 + 哪 4 个 RAG 索引会受影响」?

答「开会评估」= 没有自动血缘。

Q3 上游昨天补数了,你能不能让所有用过昨天那批数据的下游「立刻知道」?

答「群里发个通知」= 没有 Asset 级事件传播。

Q4 凌晨 3 点告警,值班同事能不能照 Runbook 把 P1 故障在 30 分钟内压下来?

答「先打电话叫人」= 没有 Runbook 或 Runbook 只是文档摆设。

Q5 过去 90 天,你能不能给出「每一次补数花了多少机器钱、改了几个分区、被谁审过」的清单?

答「查不到」= 没有补数 Audit Trail,出事追责也追不到。

MIGRATION DECISION

Airflow → Dagster 迁移路径:2026 老司机的渐进式答案



踩过的坑:不要「一次性把 200 个 DAG 迁过来」——95% 的团队会失败。一次只迁一个域,用 30-60-90 天节奏。

INDUSTRY SIGNALS · 2026

L1 收口:行业方向已经统一,但要看清谁在前面

已落地

Dagster · dbt

- Dagster Cloud Asset Graph 已是行业事实图谱
- dbt 1.9+ Materialized Lake / Cumulative Materialization
- OpenLineage 原生支持,血缘自动出
- 已被 Anthropic / Stripe / Block 大规模使用

→ 新项目从这里起步,几乎没有反对意见

在追

Snowflake · Databricks

- Snowflake Dynamic Tables(声明式 + 自动重算)
- Databricks Lakeflow(2025 重写的资产化引擎)
- 都在向「数据资产 = 一等公民」靠拢
- 平台耦合度高,跨厂场景不友好

→ 如果你深度押注某一朵云,直接用这个

在退

Airflow · 自研脚本

- Airflow 3.0 引入 Asset(Dataset)感知
- 但 Task 抽象在核心 API 仍然是一等公民
- 自研脚本 + cron + on-call 几乎不再有新项目
- 老项目继续维护,但「迁移在路上」

→ 已上线的不必拆,但不要再起新 Airflow 仓库

2024 之后,新项目几乎没人再从 Airflow 起步——不是 Airflow 不能用,是它的核心抽象错位了。

L02

LESSON 02

PARTITIONS ARE THE ATOMS

分区是资产化的原子单位——没有分区,资产化是骗子

听完 L1,很多人觉得「我们也用了 Dagster / dbt 啊,每张表都有 owner,我们也资产化了」。

我问一个问题——

2026-05-15 那一天的 customer_dim 出问题了,你能不能只重跑那一天?

答不出来的团队,资产化都只做了一半。

这一讲讲清一件事:

分区不是「为了查询快」,是让资产「在时间和维度上可寻址」——
没有分区,补数没法精准、血缘只能停在表级、回放永远是全量。

KEY THESIS

分区不是性能优化——是「时间维度上的可寻址性」

很多人把分区当查询优化,这只对一半。

在资产化数据工厂里,分区真正的作用是:

- 让你能精确指着说「就重跑 2026-05-15 这一格」
- 就补 tenant_A 在 5 月的数据,别动 tenant_B
- 下游问「你用的哪一版」,你能给出 (date, tenant, snapshot_id)

没有分区,资产化是一片连续的湖;

有了分区,资产化是一张精细的网——每一格独立物化、独立回滚、独立验证。

老司机说 → 一个常见误区:小公司想「我们数据少,不用分区」。等到第一次出事故,你会怀念那个早分好区的版本。

THREE PARTITION TYPES

三种分区类型——每种解决不同的工程问题

TIME · 时间分区

90% 默认选择

- DailyPartitions / HourlyPartitions
- 最常见、最稳定
- 解决:补数 / 回放 / 增量

关键:用 event_time,不要用 wall-clock(挂钟时间)。
event_time 是数据自己的时刻
wall-clock 是机器的时刻
迟到数据会让两者错位。

DIMENSION · 维度分区

多租户 / 灰度

- StaticPartitions(a / b / c)
- 按业务维度切
- 解决:多租户隔离 / 灰度

关键:维度值必须稳定
不能频繁加新值。
加新值意味着所有下游
都要重建分区元数据。

COMPOSITE · 复合分区

时间 × 维度

- MultiPartitionsDefinition
- 时间 × 维度笛卡尔积
- 解决:精细化运营 + 定向补数

关键:组合数别爆炸
 $5 \text{ tenant} \times 365 \text{ 天} = 1825, \text{OK}$
 $5 \text{ tenant} \times 24 \text{ 时} \times 365 = 43800$
元数据查询变慢,要权衡。

90% 的业务场景用时间分区就够。维度分区是「业务隔离需求」的产物,复合分区是「精细化运营」的产物,先有需求再上。

COMMON TRAPS

时间分区里最常踩的 4 个坑——每个都见过真实事故

坑	工程师为什么会犯	后果	正确做法
用 wall-clock 当分区键	本能、改起来快	时区错乱 / 时钟漂移 / 数据漏掉	永远用 event_time + watermark
粒度太粗(按月)	想少建几个分区	一次补数重跑整月,资源浪费	默认按天,必要时按小时
粒度太细(按 1 分钟)	想做实时	分区数爆炸、查询元数据慢	高频用流处理,分区按小时
没设 grace period	不知道上游会迟到	迟到数据漏掉,下游算错	声明 watermark_lateness 缓冲

看着最简单的「时间分区」其实最容易翻车——选错粒度、忘了水位线,2 周后业务才会发现数据少了一截。

GRANULARITY TRADE-OFFS

不同分区粒度的工程权衡 —— 别问「应该选哪个」,先问「我下游要什么」

粒度	适用场景	补数代价	元数据	推荐 SLO
小时分区	高频更新的 ticket / event	小(1 小时)	高(8760/年)	P1 · 30 分新鲜度
天分区	90% 默认	中(1 天)	中(365/年)	P1 · 4-12 小时新鲜度
周分区	聚合宽表 / 周报	高(7 天)	低	P2 · 24-48 小时新鲜度
月分区	财务月结 / 历史归档	极高(1 个月)	极低	P3 · 周度新鲜度

默认从「天分区」起步——只在两类情况偏离:高频需求往小走(小时),归档需求往大走(月)。先按天做,再调,这是 90% 团队的最优路径。

COMPOSITE · REAL CODE

复合分区实战:让「时间 × tenant」的补数代价从 $O(N)$ 降到 $O(1)$

python · pipelines/assets/ticket_silver.py

```
from dagster import (
    asset, MultiPartitionsDefinition,
    DailyPartitionsDefinition,
    StaticPartitionsDefinition,
)

TENANTS = [
    "northstar_workspace",
    "northstar_edge",
    "northstar_studio",
]

PARTITIONS = MultiPartitionsDefinition({
    "date": DailyPartitionsDefinition(
        start_date="2026-01-01"),
    "tenant": StaticPartitionsDefinition(TENANTS),
})

@asset(
    partitions_def=PARTITIONS,
    group_name="silver_support",
    tags={"criticality": "P1"},
)
def ticket_fact_by_tenant(context):
    keys = context.partition_key.keys_by_dimension
    date = keys["date"]
    tenant = keys["tenant"]

    df = load_raw_ticket(date=date, tenant=tenant)
    df = clean_and_normalize(df)

    context.add_output_metadata({
        "partition": f"{date}/{tenant}",
        "row_count": len(df),
        "snapshot_id": write_iceberg(df, date, tenant),
    })
    return df
```

INSTRUCTOR NOTES · 老司机说

这段代码的工程价值,看 3 点就够了:

- 一次补数可以精确到「2026-05-15 × northstar_edge」一个 cell——别的 tenant 不动,机器钱省一半。
- 某个租户上游出问题,只重跑那一个 cell——爆炸半径从「全表」收到「一格」。
- snapshot_id 写在 metadata 里——下游可以锁定「我用的是哪一次物化」,这就是 versioning 的底。

【注意分区数】3 tenant × 365 天 ≈ 1100,问题不大;如果再加 hour 维度,就要做分层(top-level 日,内部 hour)或者放弃 hour。

LAKEHOUSE PARTITIONS · 2026

Lakehouse 时代,分区在做什么——Iceberg / Delta / Hudi / DuckLake 的 2026 现状

引擎	分区机制	亮点	坑
Iceberg V3	Hidden Partitioning + Row Lineage	分区列不暴露 / Schema 演进零成本 / 行级血缘原生	社区版自管 metadata 文件,小文件治理要自己来
Delta Lake 4.0	Liquid Clustering	动态调整,不用 partition by 写死 / VACUUM 自动	只在 Databricks 生态体验最好,跨厂支持弱
Apache Hudi 1.x	COW / MOR + Clustering	实时 upsert 友好 / Indexing 多	运维复杂度最高,小团队劝退
DuckLake	SQL Catalog + Parquet	极简、本地起服、对中小团队和授课场景友好	2026 仍偏新,生产用要看团队工具链

选型口诀:云原生且预算够 → Databricks + Delta;开源 + 多云 → Iceberg V3;教学 / PoC → DuckLake;Hudi 留给已经在用 Hudi 的团队。

EVOLVING PARTITIONS

分区演进——业务变了,分区怎么改才不伤筋动骨

S1 只加分区维度,不改老分区

业务从单租户变多租户?新建 MultiPartitionsDefinition 并把老数据全归到 default tenant 即可,老分区一行不动。

S2 粒度变细:先双轨,再废老

从天分区变小时分区?新表 ticket_fact_hourly 双写 4 周,确认新表稳定后下游切换,老表保留只读。

S3 粒度变粗:用聚合,不重建

从小时变天?不要去 truncate,起一个新的「daily summary」资产去聚合 hourly 数据,老 hourly 当 raw 留着。

A1 硬规则:分区键一旦定下,永远不要原地改

Iceberg V3 的 partition transform 演进能减痛,但「改键」永远要走「新表 + 历史回填 + 切换」三步,没有更便宜的路。

演进的成本一半在工程,一半在协作——下游有几十个消费方时,先做好沟通(谁切、何时切、回退条件),代码反而是简单的部分。

PARTITION METADATA

每个分区都应该带的元数据 —— 这是资产化的「黑匣子」

字段	作用	不带的后果
partition_key	唯一标识这一格	无法定位补数对象
materialized_at	本次物化时间戳	不知道用的是哪次物化结果
source_snapshot_id	上游所用的 snapshot	上游变了下游不知道,出 bad case 复现不了
row_count / size_bytes	数据规模	补数前评估资源不准
status	success / failed / quarantine	下游不知道这一格能不能消费
contract_ref	所用契约版本	语义漂移无法回溯到契约源头

这套元数据 = Week 4 治理那一周「版本回滚」的全部依赖。少一个字段,事故复盘就少一条线索。

L03

LESSON 03

BACKFILL & IDEMPOTENT RETRY

回填与幂等重试:补数不是「再跑一次」,是「在历史时刻精确重建」

做数据工程最难受的一句话:「上周的数据有问题,麻烦补一下。」

听起来简单,做起来要命——

补什么?补到哪?补完下游怎么办?

消费方已经拿过去用的旧数据怎么办?

这一讲我们把「补数」从「写个脚本」升级成「工程化能力」——
4 阶段流程 + 3 层幂等 + 一份真实事故复盘 + 一笔成本账。

本节 · 10 页 · 约 25-30 分钟

KEY THESIS

补数的本质是「时间旅行」—— 精确还原系统到某个历史时刻,从那里重新走

很多人理解的补数是「再跑一次脚本」。
这是个非常危险的想法。

因为「再跑一次」用的是今天的代码 + 今天的上游 + 今天的依赖,而你要补的可能是 2 周前的数据。

2 周前的代码不一样,上游数据状态不一样,甚至外部 API 返回都不一样——
硬补一次,你会用「今天的逻辑」改写「2 周前的事实」,问题更大。

正确的补数 = 精确回到那个历史时刻,用那时的代码版本 + 那时的上游 snapshot + 重新物化。
这就叫——可重建(reproducibility)。

老司机说 → 补数最危险的一句话:"我直接 rerun 一下就好。" — 没问场景、没看快照、没通知下游,事故 80% 是这样酿出来的。

FOUR SCENARIOS

4 种典型补数场景——每种处理逻辑都不一样

场景	触发原因	关键判断	回退策略
事故补数	上游故障 / 数据错乱	原数据是否还可信?	隔离原区 + 全量重跑
字段新增补数	新增计算列	是否影响历史含义?	只增量补新字段
业务变更补数	KPI 口径调整	需不需要追溯回历史?	协调下游 + 双版本并行
上游修复补数	上游补完通知	当时下游用的是错的吗?	精确补 + 通知所有消费方

补数前必须先问:这是哪种场景?回答错了,补数就会制造更大的问题。

IDEMPOTENCY · 3 LAYERS

幂等性的 3 个层次——少一层都会在补数时炸

WRITE · 写入层

数据写入要幂等

- INSERT 改 MERGE / UPSERT
- partition_key + business_key
- 同分区多次写入结果一致

现代解药:

- Iceberg snapshot 天然幂等
- Delta MERGE + WHEN MATCHED
- dbt incremental + unique_key

没有这一层,补数 = 制造重复

SIDE-EFFECT · 副作用层

外部操作要幂等

- 发邮件 / 推送通知
- 调外部 API
- 更新缓存 / 触发下游任务

关键武器:

- idempotency_key 头
- at-least-once + 去重表

最容易漏:补数时把同一封
"账单已到账"邮件发 7 次。

DOWNSTREAM · 下游层

消费方要感知

- 下游已经消费过旧数据
- 是否需要让下游一起重跑
- 消费方锁 snapshot_id

机制:

- Watermark Event 通知
- Asset Reconciliation Sensor
- Dagster Auto-Materialize

Week 14 治理周会展开])

3 层只要漏一层,补数都会出事——99% 的补数事故,不是写入层的错,是副作用层或下游层没考虑。

IDEMPOTENCY ID STRATEGIES

幂等 ID 的 3 种策略——选错了,重试会制造重复

S1 自然主键 (Business Key)

用业务字段当主键,如 ticket_id / order_no。

优点:天然幂等。 缺点:上游改键就崩、跨域容易冲突。 适合:源头明确、业务键稳定的场景。

S2 合成主键 (UUID)

用 UUID v4 / ULID 现场生成。

优点:永不冲突。 缺点:每次重跑 ID 都不一样,完全不幂等。 适合:append-only 事件流(只入不改)。

S3 确定性合成主键 (Deterministic Hash)

用 hash(partition_key + business_key + version)。

优点:幂等 + 防冲突。 缺点:实现稍复杂、改 schema 时 hash 要做版本化。 适合:大多数生产场景的标配选择。

看到一个表 ID 是 uuid_v4(),先警惕——除非这是纯事件流,否则补数一定会重复;问设计者:你们怎么去重?如果答不上来,这就是隐患。

LATE-ARRIVING DATA

迟到数据:90% 的补数事故,根源在这里

PROBLEM · 现象

数据迟到

上游 API 5 月 18 日的数据,5 月 22 日才到达。

如果你 5 月 18 日已经 materialize 了那一天的分区,这批数据要算今天的还是昨天的?

SIGNAL · 水位线

Watermark

声明 `watermark_lateness = 7d`:

「event_time 在 7 天内的迟到数据,归入它原本所属的分区」

超过 7 天 → 进 quarantine。

STRATEGY · 落分区

回填那一格

迟到数据到达 → 触发那一天的分区重物化(不是新建,是 overwrite)。

Dagster Auto-Materialize 或 dbt incremental + filter 都能实现。

GUARD · 阈值

占比报警

迟到数据占当日总量 > 5% → 报警。

说明上游有系统性延迟,要从根上修,不只是补数据。

水位线不是「写一行 `watermark = 7d`」就完了——下游所有 SLO 公式都要把 7 天延迟算进去,产品和业务那边必须沟通到。

BACKFILL WORKFLOW

一次合格的补数 —— 4 个阶段,每个阶段都有产物(Artifact)



工程团队最容易跳过的是「评估」和「验证」——但这两步出问题,代价远大于补数本身。

DAGSTER BACKFILL · REAL

一次补数:CLI + Python + 必填的 tag——别让 audit 追溯不到人

bash / python · pipelines/jobs/backfill_ticket.py

1) CLI 方式:点选范围,声明原因和工单号

```
dagster job backfill \  
  --job ticket_silver_job \  
  --partition "2026-05-01:2026-05-07" \  
  --tenant northstar_workspace,northstar_edge \  
  --tags "reason=upstream_fix,\  
        ticket=INC-2026-0512,\  
        approver=zengdan@,\  
        risk_level=P1"
```

2) Python 方式:可编程触发(适合自动化补数)

from dagster import materialize, MultiPartitionKey

```
result = materialize(  
    assets=[ticket_fact_by_tenant],  
    partition_key=MultiPartitionKey({  
        "date": "2026-05-15",  
        "tenant": "northstar_edge",  
    }),  
    tags={  
        "backfill_run_id": "bf-20260518-001",  
        "approver": "team:support-platform",  
        "reason": "upstream_fix",  
        "source_snapshot_at": "2026-05-22T03:12:00Z",  
    },  
)
```

Dagster 自动做的事:

- # · 展开分区清单(笛卡尔积、去重已成功)
- # · 按 max_concurrent 控制并发(别压垮上游)
- # · 失败隔离 + 重试(只重跑失败 cell)
- # · 上下游传播(下游自动感知,触发 reconcile)
- # · 物化历史:materialized_at / snapshot_id / row_count

INSTRUCTOR NOTES · 老司机说

看 4 个工程点:

- --partition 用区间,Dagster 自动展开,你不用写 for-loop。
- --tags 必填 reason / ticket / approver,审计能追到人——少一个 tag PR 不该过。
- source_snapshot_at 是「时间旅行」的关键:告诉系统按那个时刻的上游 snapshot 跑。
- 编程触发用于自动化(比如检测到上游补数完成,自动起下游 backfill)——这是从「人工」走向「半自动 Runbook」的桥。

ASSESSMENT TEMPLATE

补数评估单 5 问 —— 这一页直接用作 PR 模板

评估项	具体问题	不评估的后果
范围	哪些分区?哪些 tenant?多大批?	补漏了 / 补多了 / 影响别的租户
资源	需要多少 CPU / 多少 GPU / 多少时间?	挤占在线服务资源,引发二次事故
影响	哪些下游已经消费过这批旧数据?	下游继续用错数据生产新错
策略	reject + replay 还是增量修补?	选错策略 = 修不好,反复返工
回滚	失败了怎么回到补数前的状态?	补数失败 = 数据进半成品状态

这张评估单不是流程文档,是 PR 模板——每次发起 backfill PR 必须填,reviewer 看不到这张表,直接 close。

WAR ROOM · REAL STORY

一次真实事故:补 7 天数据,3 个小时和 3 个深坑

TIMELINE

T+0	上游 API 返回 schema 漂移	2026-05-22 09:15 · contract check 报警,17 个分区进 quarantine
T+30m	评估 → 影响 7 天 × 3 tenant	21 个 cell,下游 14 个消费方,5 个已发出报表
T+1h	审批 → 主负责人 + 下游 owners 同意	PR + 群周知 + Slack 工作流通知 7 个 RAG 索引
T+1h30	执行 → Dagster Backfill 起 9 路并发	reject + replay,iceberg.rollback_to_snapshot 兜底
T+2h45	验证 → 21 个 cell 全部 SUCCESS	对账 query 通过,下游 KPI 偏差 < 0.3%
T+3h	通知下游 + 关 incident	发 Watermark Event,RAG 索引自动重建

事后复盘:3 个差点搞砸的深坑

坑 1 副作用没幂等 "账单已修复"邮件给同一个客户发了 5 次。 FIX:补数前关掉所有外发副作用 (toggle)。	坑 2 RAG 索引滞后 索引还在用旧 chunk,客服回答仍然错。 FIX:发 Watermark Event,RAG 自动 reindex。	坑 3 下游报表对齐 5 月业绩报表已经出,补数后差了 0.3%。 FIX:出补丁报表 + 通知接收方更新。
---	---	---

THE COST OF BACKFILL

补数的成本账——别让一次"小补"变成季度账单的大坑

一次中等规模 backfill 的真实账本(对照案例 · 3 tenant × 7 天)

项	估算	说明
Compute 直接消耗	~ ¥2,400	21 个 cell × 平均 8 分钟 × 中型 Spark cluster
上游 API 重拉	~ ¥800	21 次重新拉取原始数据,部分供应商按调用计费
存储/快照膨胀	~ ¥300	新 snapshot + 老 snapshot 同时保留 30 天
下游 RAG 重索引	~ ¥1,500	14 个索引重建,embedding API 按 token 计费
工程人时	~ ¥3,200	2 个工程师 × 3 小时 + 1 位 reviewer
—— 总计	~ ¥8,200	一次「3 小时小补」实际成本 ≈ 一个工程师一周薪资

老司机怎么把账压一半：

- 评估单上必须写「能不能不补」——大量「补数」其实是「重新声明 contract + 加 reject 规则」就够。
- 用 reject + replay 而不是 truncate + reload——只动错的格,不动好的。
- 下游索引重建走「增量 reindex」而不是「全量 reindex」,RAG 成本能砍 70%。
- 高频 backfill 集中到夜间低价时段(部分云有低峰价),账单能再降 20-30%。

L04

LESSON 04

LINEAGE · END-TO-END

全链路血缘:数据系统的 X 光机

想象一个真实场景——

产品经理跑过来:「我们改了客户分级字段,影响哪些报表?哪些 AI 的回答?」

没有血缘的团队:开会 3 小时、问 5 个组、还可能漏掉。

有血缘的团队:渲染一张 Asset Graph,30 秒给出完整影响范围。

这一讲讲清:

血缘不是好看的图,是数据系统的 X 光机——

能不能用、用得好不好,决定了你的团队能跑多快。

KEY THESIS

血缘不是「文档」,是「运行时自动生成的拓扑」—— 文档会撒谎,血缘不会

过去 20 年企业都有「数据字典」「血缘文档」——
90% 的团队都失败了,因为人工维护的血缘永远赶不上代码变化。

现代数据栈对血缘的根本性突破:

血缘不再是「人写的文档」,而是「编排引擎跑的时候自动产生的元数据」。
代码改了,血缘自动更新——文档不会过期,因为它本身就不是文档。

一个心智转换——
不要再问「我们的血缘文档是谁负责维护的?」
正确的问题是「我们的血缘事件流是从哪个 emitter 发的?数据存到哪个 backend?」

老司机说 → 看到「血缘文档」这四个字就要警惕——再认真的人,半年后这份文档都会变成“看着像那么回事”的废纸。

THREE LEVELS

血缘的 3 个粒度——每一层解决不同问题

TABLE · 表级

入门档

- A.table → B.table
- 最容易做,工具支持最广
- 解决:粗粒度影响分析

工具:dbt-docs / DataHub

局限:

- 改一个字段,说不清
哪些下游会受影响
- BI 时代够用,AI 时代不够

COLUMN · 字段级

生产档

- A.col1 → B.col2
- 工程难度大 10 倍
- 解决:精细影响分析

工具:OpenLineage SQL Facet
sqllineage / Marquez

局限:

- 复杂 SQL(window / UDF)
解析仍是难题

ROW · 行级

硬核档

- 某一行的源头
- 性能成本极高
- 解决:bad case 追溯

工具:Iceberg V3 Row Lineage
OpenLineage + audit

局限:

- 全量行级开销过大
- 通常只在抽样 + 关键域上做

2026 的及格线:表级 100% + 字段级覆盖核心域 + 行级在关键 AI 输出链路上做。Iceberg V3 的 Row Lineage 让行级第一次有了规模化可能。

FIVE USE CASES

血缘的 5 大场景——每一个都直接决定 MTTR

U1 影响分析(Impact Analysis)

上游改 schema 前,先渲染影响范围:哪些 asset / dashboard / RAG 索引会受影响。原来开会评估,现在按一下按钮。

U2 故障定位(Root Cause)

下游 KPI 异常,沿血缘逆向追溯,找到第一个出问题的资产分区。原来 3 小时,现在 5 分钟。

U3 合规审计(Compliance)

审计员问:「这个客户字段在哪 7 个系统里使用过?」血缘直接出报告。GDPR / CCPA + AI Act 都靠它。

U4 查询优化(Query Optimization)

看哪些下游都在消费一张「巨表」——精准物化中间层,避免下游重复 join。这是给 FinOps 的礼物。

U5 Bad Case 复现(Reproducibility)

AI 回答错了——血缘 + snapshot ID 让你回到那一刻,看那时的数据、复现整个推理过程。这是 AI 治理的根。

OPENLINEAGE MODEL

OpenLineage 的 4 个核心对象——理解这 4 个,就理解了现代血缘

对象	含义	主要字段	什么时候产生
Job	一段可重复执行的代码	name / namespace / source	代码定义时声明
Run	一次具体执行	run_id / start / end / state	每次运行时生成
Dataset	一份数据集(表/文件/流)	name / namespace / schema	输入或输出时声明
Facet	附加元数据扩展	任意结构化扩展信息	按需附加到上面 3 类对象

这 4 个对象一组合,就能描述「某个时刻、某段代码用了哪些数据、产生了哪些数据」——这就是血缘的全部。

REAL EVENT

一个真实的 OpenLineage 事件 —— 你不需要写它,Dagster 跑完自动发

json · emitted by dagster-openlineage

```
{
  "eventType": "COMPLETE",
  "eventTime": "2026-05-22T11:32:15Z",
  "run": {
    "runId": "bf-20260522-001",
    "facets": {
      "parent": { "job": "backfill_ticket_silver" },
      "nominalTime": {
        "nominalStartTime": "2026-05-15T00:00:00Z"
      }
    }
  },
  "job": {
    "namespace": "omnisupport.support_ops",
    "name": "ticket_silver.materialize"
  },
  "inputs": [{
    "namespace": "omnisupport.support_ops.bronze",
    "name": "raw_ticket",
    "facets": {
      "schema": { "fields": [...] },
      "version": { "datasetVersion": "snap_42" }
    }
  }],
  "outputs": [{
    "namespace": "omnisupport.support_ops.silver",
    "name": "ticket_fact",
    "facets": {
      "schema": { "fields": [...] },
      "outputStatistics": { "rowCount": 12480 },
      "columnLineage": { "fields": {...} }
    }
  }],
}
```

INSTRUCTOR NOTES · 老司机说

这就是一次资产物化自动产生的 OpenLineage 事件。

· inputs / outputs 自动带「上游用的哪个 snapshot」+「输出了什么」——这就是血缘。

· facets 是扩展点:schema、版本、统计、columnLineage 都挂这里——它会随生态长大。

· 所有事件汇集到血缘后端(Marquez / DataHub / OpenMetadata)后,形成实时资产图。

关键:这是机器产生的,不是人写的——所以它不会过期。「血缘文档」时代结束了,「血缘事件流」时代开始了。

AI-NATIVE LINEAGE

AI 时代,血缘要管到哪里?边界已经被推远了

过去:血缘 = 「表 → 表 → 表」。

AI 时代:血缘要纳入 vector store / prompt / model / eval set / agent decision——
只有这样,bad case 复现才走得通,审计才答得上来。



AI-Native 血缘的 4 个工程动作:

- 每个 chunk 带 source_asset_id + source_snapshot_id —— bad case 复现的根
- Vector store 的 collection 视为「资产」纳入 Asset Graph,每次重建都发 OpenLineage 事件
- Prompt template 走 Git + version_tag,推理时记录到 Trace,血缘里能看到「这次回答用的是 prompt v17」
- Eval set 也是资产 —— 在血缘里能看到「v17 prompt 在 eval_set_v3 上跑分多少」

COLUMN LINEAGE · REAL TRAPS

字段级血缘的现实坑——SQL Parser 撑不住的地方,你要自己来

T1 Window function 解析盲区

sqllineage 这类静态解析器在 ROW_NUMBER() OVER(...) / LAG / LEAD 这种里几乎都会丢字段。
Workaround:核心域改用 dbt model 显式声明 columns,把字段血缘下沉到 model 元数据。

T2 UDF 直接断链

一个 Python UDF / Spark Function 内部跑了什么,SQL parser 看不到。
Workaround:UDF 显式声明 inputs / outputs 元数据,在 OpenLineage 事件里用 ExtractionFacet 补上。

T3 JSON / 半结构化字段

json_extract(payload, "\$.user.id") 这种,parser 经常只能解到 payload,看不到内部路径。
Workaround:把 JSON 字段先拍平(structify)到独立列,血缘自然恢复。

T4 跨引擎(SQL → Spark → ML)

一条管线跨 3 个引擎,事件标准不一致 → 血缘断链。
Workaround:统一 OpenLineage emitter,所有引擎都发同一种事件——这是 2026 年新项目的硬要求。

接受现实:字段级血缘不可能 100% 自动,90% 自动 + 10% 关键域手工补,是 2026 年的最佳实践。

INTEGRATION & SELECTION

Dagster + OpenLineage 集成 · 血缘后端选型 2026

python · pipelines/lineage/setup.py

```
from dagster import Definitions, EnvVar
from dagster_openlineage import (
    openlineage_resource,
)

defs = Definitions(
    assets=[
        ticket_fact_by_tenant,
        ticket_kpi_mart,
        kb_serving_chunks,
        kb_vector_index,
    ],
    resources={
        "openlineage": openlineage_resource.configured({
            "transport": {
                "type": "http",
                "url": EnvVar("OPENLINEAGE_URL"),
                "endpoint": "/api/v1/lineage",
            },
            "namespace": "omnisupport",
            "extractors": ["sql", "schema", "stats",
                           "column", "vector_store"],
        }),
    },
)
```

之后:所有 asset 物化时自动发血缘事件
asset 函数本身不需要写任何额外代码
DataHub UI 上能看到:
ticket_fact_by_tenant
└— upstream: raw_ticket@snap_42
└— downstream:
support_kpi_mart, kb_vector_index

血缘后端选型 · 2026

产品	定位	适合
Marquez	开源最小可用	小团队 / PoC
DataHub	血缘 + 目录 + 治理	中大型组织
OpenMetadata	统一元数据 + 治理	需要治理协作
Atlan	体验最现代	预算充足的商业团队
Unity Catalog	Databricks 原生	深度押 Databricks

默认路径:新项目 OpenLineage + DataHub;
如果在 Databricks 生态 → Unity Catalog 优先。

L05

LESSON 05

RUNBOOK · INCIDENT & COLLABORATION

应急手册 + 协作规范:把经验沉淀成肌肉记忆

前 4 节我们建好了数据工厂的核心能力——
Asset / Partition / Backfill / Lineage。

这一节做两件事：

1. 把这些能力组装成一份能在凌晨两点照搬执行的 Runbook;
2. 把团队协作的规则定下来——命名 / Owner / 变更隔离 / Game Day。

这一讲完了,你不是「我会用 Dagster」,而是「我和我的团队能稳稳地运行一座数据工厂」。

KEY THESIS

Runbook 不是文档,是「事故压力下也能稳定执行的脚本」

我见过最差的 Runbook 长这样——

「检查上游数据是否正常。」

什么叫正常?查哪些字段?阈值多少?凌晨两点没人看得懂。

好的 Runbook 长这样——

「执行 `dagster job status ticket_silver_job --partition 2026-05-15;`
如果 status 是 FAILED,跳到第 3 步。」

Runbook 的核心不是「描述」,是「指令」——一个能在凌晨两点照搬执行的脚本。

老司机说 → 判断一个团队是否真有运维能力,看 Runbook——看不到具体命令的 Runbook,等于没有。

ANTI-PATTERNS

Runbook 失败的 6 大反模式 —— 你的 Runbook 中招几个？

反模式	具体表现	后果	正确做法
抽象描述	"检查上游是否正常"	凌晨两点没人看得懂	给具体命令 + 期望输出
口口相传	"问老 X 就行"	老 X 离职就完蛋	强制每个故障写 Runbook
只在事故后写	复盘时补 Runbook	凭记忆不准,有遗漏	事前演练 + 事后修订
没人测试	写完没人跑过	真用时命令早废了	每季度 Game Day 演练
没有验证步骤	操作完不知道好不好	修了还在出问题	每个动作配验证命令
不连接监控	看不到当前状态	凭感觉判断	Runbook 第一步看 Grafana

Runbook 不是写出来的,是「演练 + 反复修订」出来的——第一版永远是错的。

FIVE ELEMENTS

一份合格 Runbook 的 5 个必备要素 —— 缺一个就用不起来

R1 Symptoms · 症状

怎么发现这是这类故障?告警是什么?面板长什么样?给出可观察的判定特征——截图 + 阈值,不是用文字描述。

R2 Diagnosis · 排查

症状到根因的判断流程图——决策树形态。哪条分支查什么、跑哪个命令、期望输出是什么。

R3 Action · 操作

修复操作的具体命令——每一行都能复制粘贴。不要写"重启服务",要写"systemctl restart dagster-daemon"。

R4 Verification · 验证

修完怎么确认好了?查哪些指标、跑哪些验证命令、期望多久内恢复正常。没有验证 = 修了等于没修。

R5 Postmortem · 复盘

事故复盘模板:发生了什么 / 影响范围 / 根因 / 改进项 / 是否升级 Runbook。每次事故必须产出,沉到知识库。

REAL RUNBOOK

真实模板:ticket_silver 物化失败 · schema 漂移

markdown · runbooks/ticket_silver_schema_drift.md

Runbook · ticket_silver schema drift

SYMPTOMS / 症状判定

- Dagit UI: ticket_fact_by_tenant FAILED
- 错误信息含: "ContractValidationError"
- Slack #data-alert 收到 P1 告警

DIAGNOSIS / 排查

1. dagster asset-check status ticket_fact_by_tenant
 - ↳ contract violation: 跳 ACTION-A
 - ↳ schema mismatch: 跳 ACTION-B
 - ↳ 其他: 升级 on-call senior

ACTION-A · 契约违反

1. pytest tests/contract/test_ticket.py -v
2. slack @support-platform-oncall
3. additive 变更 → 升级 contract
4. breaking 变更 → quarantine 流程

VERIFICATION / 验证

- dagster job execute ticket_silver_job \ --partition \$(date -u +%F)
- 预期: 5 分钟内 SUCCESS
- Grafana: support_ops.silver.lag_min < 30

POSTMORTEM / 复盘

- 见 postmortems/template.md
- 必填: 时间 / 影响 / 根因 / 改进项

3 档成熟度——你团队在哪一档

L1 · Manual

人工执行

Markdown / Confluence, 每步明确命令。
所有团队的起步, 不要跳级。

L2 · Semi-Auto

半自动

关键步骤变 CLI / Dagster Job。
人工触发 + 流程化, 失败回退人介入。

L3 · Fully-Auto

全自动

Skill Pack / Self-healing。
检测+决策+执行闭环, 只看结果。

COLLAB · NAMING & OWNERSHIP

协作规范第一步:命名、Ownership、SLO 的团队约定

资产命名 · 团队约定

一份命名规范的 4 条铁律

- 1) 命名空间:{domain}.{layer}.{name}
✓ support_ops.silver.ticket_fact
✗ silver_ticket / ticket_v2_final
- 2) layer 仅四档:bronze / silver / gold / serving
不要造新词(没有 "platinum")
- 3) 名词单数 + 蛇形:不写复数,不写驼峰
✓ ticket_fact ✗ tickets_facts / TicketFact
- 4) 永远不要把版本写在 name 里
✓ ticket_fact(版本走 snapshot_id)
✗ ticket_fact_v2 / ticket_fact_new

Ownership + SLO · 团队约定

4 条不可妥协的硬规则

- ① 每个资产 ≥ 1 个 Owner(team-level,不是个人)
tag:owners=["team:support-platform"]
- ② 每个 P1 资产必须声明 FreshnessPolicy
maximum_lag_minutes / cron_schedule
告警自动 routing 到 Owner team
- ③ criticality 强制三档:P1 / P2 / P3
P1 $\rightarrow$ 30 分新鲜度 + 立刻响应
P2 $\rightarrow$ 4-12 小时 + 工作时间响应
P3 $\rightarrow$ 周度 + 尽力而为
- ④ 任何「无主资产」不准上 prod
CI 检查:asset 没声明 owner $\rightarrow$ fail

CHANGE ISOLATION

变更隔离:让多人同时改资产不互踩——Branch Deployments + slim CI

90% 的「我改完上线发现你也改了」事故,根因是没做变更隔离。解药是把「数据栈」当代码栈管——每个 PR 起一个独立的数据沙盒,合到 main 才生效。

Dagster · Branch Deployments

每个 PR 自动起一个隔离的 Dagster 部署

- 开 PR → Dagster Cloud 自动起 branch deployment
- 同一份代码跑在「沙盒 schema」上,不动 prod
- 沙盒里能跑 backfill / 跑 asset check / 看血缘
- merge 到 main → 自动 promote 到 prod
- 关 PR → 沙盒资源自动回收

工程价值:

- 两个团队同时改同一张表,各自有沙盒,互不踩
- reviewer 可以直接在沙盒里看效果
- 出问题:retract PR 即可,prod 一行不动

dbt · slim CI

只跑 PR 真正影响到的 models,而不是全量

- state:modified+ → 只跑被改的 model + 下游
- defer → 上游引用直接指 prod state
- 一个 PR 的 CI 时间从 30 分钟 → 3 分钟
- 配合 dbt build → test 内置在同一次 CI

命令(放进 GitHub Action):

```
dbt build \  
  --select state:modified+ \  
  --defer \  
  --state ./prod-state
```

工程价值:

- 工程师不用等长 CI → 迭代节奏 5x
- 资源消耗骤降,FinOps 也开心

GAME DAY

Game Day 演练:把 Runbook 跑成肌肉记忆

Runbook 写完没有 Game Day,就跟健身办了卡不去一样。
一年至少 4 次 Game Day,把 Runbook 在「假事故」里跑一遍——这是 Runbook 唯一可信的检验方式。

STEP 1 · 选场景

场景库随机抽

- 从历史事故 + 风险清单里随机抽一个:
- 上游 schema 漂移
 - 某 tenant 数据漏
 - iceberg metadata corrupt
 - 主备节点同时挂

STEP 2 · 选演员

随机抽人

- on-call 名单里随机抽 1 人当主力
+ 1 人当观察员。
- 抽到 senior 也照样跑,不能换人——
Game Day 不是让最强的人来跑。

STEP 3 · 真实演

只用 Runbook

- 观察员只能记录,不能提示。
主力照 Runbook 命令一行行执行。
- 卡住的地方 → 就是 Runbook 的 bug。
每个 bug 立刻记录到 issue。

STEP 4 · 复盘升级

当晚出修订

- 当晚 1 小时 复盘:
- Runbook 哪一步不顺?
 - 命令是不是已经废了?
 - 监控有没有缺?
- 当晚提 PR,3 天内合并。

Google 叫 Wheel of Misfortune,Netflix 叫 Chaos Monkey ——本质都是同一件事:让事故在「可控时刻」发生,Runbook 才不会撒谎。

POSTMORTEM TEMPLATE

事故复盘模板:一份能让团队真的学到东西的 Postmortem

一份好的 Postmortem 不是「回顾故障」,是「让同类问题不再发生」——少了任何一节,事故都白出了。

① 摘要(TL;DR)

3 句话讲清楚:发生了什么、影响了谁、修了多久。让没参与的人 30 秒读懂。

② 影响范围

受影响的 asset / 下游 / 客户数 / SLO 违反时长。要带数字,不要写"少量"。

③ 事件时间线

T+0 → T+N 完整时间线。每一行写「谁做了什么,看到了什么」。这是事后所有改进的素材。

④ 根因分析(5 Why)

不要停在「网络抖了」「上游错了」——继续 ask why 直到看见组织/设计问题。

⑤ 做得好的地方

不仅记录失败,也记录响应快的人 / 起作用的工具——固化下来。

⑥ 做得不好的地方

坦诚——只有诚实复盘的团队才会进步。但只对事不对人(Blameless)。

⑦ 改进项 (Action Items)

每一项 → 责任人 + DDL + 是否完成。这是 Postmortem 唯一的产出。

⑧ Runbook 升级建议

本次事故是否升级 Runbook?如果是,新版 PR 链接放这。

RUNBOOK → SKILL PACK

Runbook 的下一站:Skill Pack——给 Agent 也能执行的「可调用知识」

维度	Runbook(人执行)	Skill Pack(人 + Agent 都能执行)
形式	单个 Markdown 文档	目录 + SKILL.md + scripts/ + assets/
触发	人收告警手动打开	description 字段供 Agent 自动匹配
执行	人复制命令逐步执行	脚本可被 Agent 参数化调用
验证	人看结果判断	scripts 内置验证 + 结构化输出
约束	靠人理解上下文	YAML frontmatter 声明输入 / 输出 / 风险
版本	Git 跟 Markdown	完整目录纳入 release manifest

Week 9 会专门展开 Anthropic Agent Skills 标准——届时你今天写的 Runbook,会被升级成「Agent 触手可及的能力」。

WHO IS DOING THIS

这条演进路径——大厂已经在走

Google SRE

Wheel of Misfortune

- 随机抽人 + 随机故障
- 必须照 Runbook 跑通
- 跑不通 = Runbook 是 bug

- 保证 Runbook 长期可用
- 新人 onboarding 直接接手

Anthropic

Agent Skills Spec

- 把工程规范封装为 Skill
- Claude / Codex 按需发现加载
- YAML frontmatter 声明边界

- Runbook → 自动化的桥
- Week 9 详细展开

Netflix

Chaos + Runbook Driven

- Chaos Monkey 主动制造故障
- 强制 Runbook 演练
- 没 Runbook 的服务不准上线

- 文化:事故是常态
- Runbook 是工程一等公民

Runbook 在大厂从「补救文档」升级成「工程一等公民」——这是 SRE 文化的核心成果,也是 AI 数据工程绕不开的一步。

你已经造好了一座"资产化数据工厂"

Asset

范式转换

声明式资产 + 4 条军规

Partition

原子化

时间×维度 + Lakehouse 分工

Backfill

工程化补数

4 阶段 + 3 层幂等 + 成本账

Lineage

运行时血缘

OpenLineage + AI-Native 边界

Runbook + Collab

协作演练

5 要素 + Game Day + Postmortem

本周交付物(已 push 至 GitHub omnisupport-copilot 仓库)

- pipelines/assets/*.py — 资产定义
- pipelines/partitions.py — 分区声明
- pipelines/jobs/backfill_*.py — 补数 Job
- pipelines/lineage/setup.py — OpenLineage 集成
- runbooks/*.md — Runbook 模板 + 真实样例
- postmortems/template.md — 事故复盘模板
- .github/workflows/branch_deploy.yml — Branch Deployments

